Software Architecture Project

Project Overview

- In this project, a teamwork group will design and implement a fully functioning backend application that exposes RESTful API.
- The application must follow the principles of Clean Architecture and good RESTful design.
- The application will be completed in groups of three or four students.
- The teamwork group may choose the topic of the backend yourselves, but the topic and the planned functionality must be approved by the instructors before implementation begins.
- The topic of your backend application, along with its functional and non-functional requirements, must be submitted as a Word document to the "Teamwork Topics for Approval" submission box for instructor approval by **Friday, 9 October 2026, at 16:00.**
- It is sufficient for one member of the group to submit the document.
- The evaluation of the teamwork happens by demonstrating the functioning backend to the instructors and the other students **on site on Monday, 14 December 2026**. Also, the related architectural documentation and the source code will be reviewed during this event.
- The application must demonstrate understanding of software architecture, maintainability, separation of concerns, and professional software development practices.
- The application must persist data in a database and expose functionality through a documented REST API. The API will be tested and demonstrated using Swagger/OpenAPI.
- The project should reflect the architectural concepts covered during the course, including Clean Architecture principles, SOLID design principles, dependency management, and maintainable code structure.

General Technical Requirements

- Implement a RESTful API with CRUD operations, proper validation, error handling, and appropriate HTTP status codes.
- Use a database for persistent data storage and separate database access from business logic.
- Follow Clean Architecture principles and separate the solution into API, Application, Domain, and Infrastructure layers.
- Business logic must not be implemented directly in controllers.

- Apply SOLID principles, Dependency Injection, interfaces, and abstractions where appropriate.
- Swagger/OpenAPI must be implemented and available for demonstrating and testing the API.

Assessment Criteria (30 points)

The number of personal points awarded to team members may vary. A passive member may receive no points at all. An especially active and cooperative member, as identified by the other team members, may receive a few additional points.

All group members must actively participate in both architectural design and programming. Each member must have at least five commits in the project's Git repository, and these commits must be distributed over at least three different weeks. At least two of these commits must be related to programming and two to architectural design.

Please also note that all team members must be present at the presentation event. An absence from presentation will result in a 25% deduction of accumulated points for the absentee.

The teamwork will not be evaluated if the following criteria are not met:

- The backend implements the approved topic.
- Git is used throughout the development process.
- In addition to the source code, the Git repository contains architecture-related documentation.
- The Git repository contains also documentation of AI use during the development process.
- The presentation is given.

Assessment Criteria table

Category	Criterion	Points
A. Clean architecture	The architecture of the application follows the dependency rule of Clean Architecture.	2
	The project structure of the application reflects Clean Architecture.	2
	The application layer contains use cases, application services, validation, and error handling using result types or equivalent techniques.	2
	The interface adapters layer converts data between external interfaces and application use cases and does not contain business logic.	1

	The frameworks and drivers layers contains infrastructure concerns such as databases, web frameworks, and external services while keeping them isolated from business rules.	1
	Subtotal	**8**
B. The quality of architecture and design principles	The application demonstrates at least two different SOLID principles that have been successfully applied.	2
	Dependency Injection is used appropriately to decouple application components.	1
	The architecture supports maintainability and future extension with minimal impact on existing code.	3
	Static typing is applied consistently.	2
	Subtotal	**8**
C. Domain-based design and code quality	Variable, function, type, and class names follow the domain terminology and communicate their responsibilities clearly.	1
	The domain layer contains entities, values, …	1
	Subtotal	**2**
D. Interfaces and integrations	The REST API is designed and implemented according to RESTful API best practices.	2
	Interactive Swagger/Open API REST documentation is implemented and functional.	2
	Subtotal	**4**
E. Documentation & Trade-off Analysis	Layer component diagrams are created and accurate. In addition to this, two other architectural diagrams are created.	3
	Main architectural decisions made during the project are documented and justified.	3
	Architectural decisions include discussion of alternatives considered and trade-offs	1
	The benefits and drawbacks of applying Clean Architecture to the application and its development are discussed and evaluated.	2
	Subtotal	**9**
F. Testing and QA	The are examples of implemented unit and integration tests.	1
	The testing practice follows the principles of Clean Architecture	2
	Subtotal	**3**
	TOTAL	**33**

Deliverables

- Complete source code in a GitHub repository.
- README-file including project overview, installation instructions, setup instructions, running instructions, API access information, assumptions, and limitations.
- Architecture documentation including a system architecture description and component or layer diagram.
- Trade-off analysis (approximately 1-2 pages) discussing alternative solutions, architectural decisions, and key trade-offs.
- Swagger/OpenAPI documentation available and functional.

Possible deductions

Major Fault	**Final Score Deduction Percentage**
The application does not start or crashes repeatedly during execution.	-25%
The application contains significant errors or behaves incorrectly.	-10%
Core use cases are not implemented.	-25%
The designed and implemented application does not correspond to the approved project topic.	-100 % | Failed assignment
The possible external tutorials, projects or other resources are used as a model and not mentioned.	-50%
The possible external tutorials, projects or other resources are used almost as such.	-100 % | Failed assignment

Other information



You are allowed to use AI during the different phases of your backend application development process. However, you must document the use of AI.